

Local LLM Routing & Escalation System

Implementation Specification v1.0

Status

Build this as a small, reliable local service for use with Claude Code.

The system must provide:

- local-first inference
- automatic escalation from a fast local model to a stronger local model
- optional escalation to OpenRouter
- explicit manual model selection
- Claude Code integration
- session/task state
- observable escalation signals
- OpenAI-compatible API
- streaming
- tool-call transparency
- health checks
- metrics
- strong cloud-cost safeguards

The system must be simple enough to understand and debug.

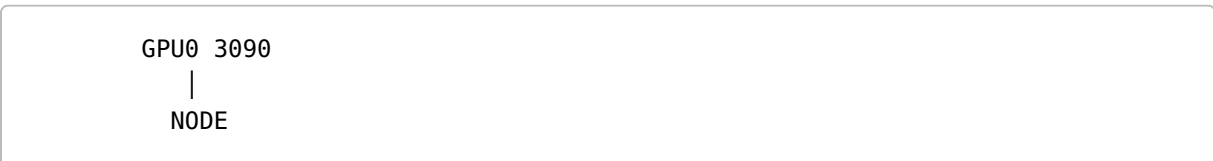
Do not over-engineer v1.

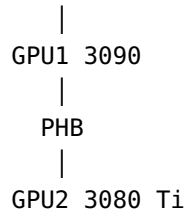
1. Hardware Context

Target machine:

GPU0: NVIDIA RTX 3090 24 GB
GPU1: NVIDIA RTX 3090 24 GB
GPU2: NVIDIA RTX 3080 Ti 12 GB

GPU topology:





GPU0 and GPU1 are the preferred pair for the larger local model.

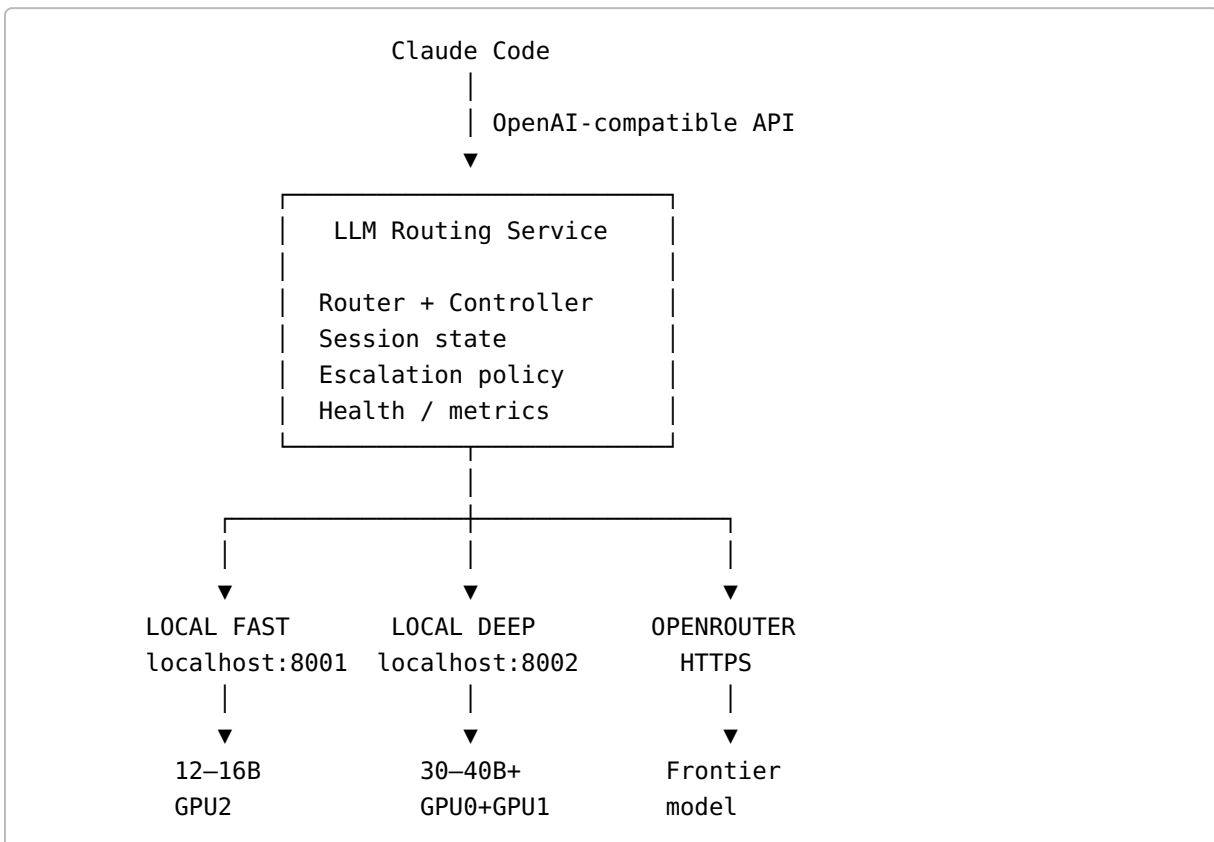
GPU2 is intended for the small/fast model.

Do not assume that all three GPUs form a single efficient inference pool.

The router must remain completely model- and GPU-agnostic.

2. Intended Runtime Topology

The intended deployment is:



Both local models should normally remain running.

Do NOT implement model unloading/reloading in v1.

The purpose of this project is routing, not GPU lifecycle management.

3. Core Design Principle

Separate these responsibilities:

```
ROUTER
  ↓
"Where should this request go?"

CONTROLLER
  ↓
"Has this task become difficult enough to escalate?"

CLAUDE CODE
  ↓
"How should the agent behave and when should it request escalation?"

BACKENDS
  ↓
"Execute the actual inference request."
```

Do not combine all of these into one giant component.

4. Three Model Tiers

The system has three logical tiers.

Tier 1 — fast

Alias:

```
local-fast
```

Purpose:

- normal coding
- file editing

- repository exploration
- simple debugging
- simple tests
- straightforward implementation
- shell/tool operations
- routine refactoring

Expected model:

12–16B

Backend:

llama-server :8001

GPU:

RTX 3080 Ti

Tier 2 — deep

Alias:

local-deep

Purpose:

- difficult debugging
- architectural reasoning
- complicated refactoring
- subtle interactions
- difficult test failures
- difficult code generation
- deeper analysis

Expected model:

~30–40B or similar

Backend:

```
llama-server :8002
```

GPU:

```
RTX 3090 + RTX 3090
```

Tier 3 — frontier

Alias:

```
frontier
```

Purpose:

- extremely difficult reasoning
- unresolved problems after local escalation
- architecture requiring stronger reasoning
- difficult security analysis
- tasks where local models repeatedly fail

Backend:

```
OpenRouter
```

The exact frontier model is configuration-driven.

Never hard-code a particular provider model.

5. Router API

Expose an OpenAI-compatible API.

Primary endpoint:

```
POST /v1/chat/completions
```

Additional endpoints:

```
GET /v1/models
GET /health
GET /health/backends
GET /metrics
```

The client should be able to treat this service like an OpenAI-compatible API server.

6. Logical Model Names

Expose:

```
local-fast
local-deep
frontier
auto
```

Example:

```
{
  "model": "local-fast"
}
```

routes directly to the fast backend.

```
{
  "model": "local-deep"
}
```

routes directly to the deep backend.

```
{
  "model": "frontier"
}
```

routes to OpenRouter.

```
{  
  "model": "auto"  
}
```

activates automatic routing/escalation.

7. Explicit Routing Priority

Routing priority must be:

1. explicit escalation/control header
2. explicit model
3. session route
4. automatic policy
5. configured default

For example:

```
X-LLM-Route: deep
```

must force the request to the deep backend.

Supported values:

```
fast  
deep  
frontier  
auto
```

8. Configuration

Use YAML.

Example:

```
server:  
  host: "127.0.0.1"
```

```
port: 8000

defaults:
  route: "auto"

backends:

  fast:
    type: "openai_compatible"
    name: "local-fast"
    base_url: "http://127.0.0.1:8001/v1"
    model: "local-fast"
    timeout_seconds: 300

  deep:
    type: "openai_compatible"
    name: "local-deep"
    base_url: "http://127.0.0.1:8002/v1"
    model: "local-deep"
    timeout_seconds: 600

  frontier:
    type: "openrouter"
    name: "frontier"
    base_url: "https://openrouter.ai/api/v1"
    model: "MODEL_NAME"
    api_key_env: "OPENROUTER_API_KEY"
    timeout_seconds: 900

routing:

  default: "fast"

  escalation:
    enabled: true

    max_fast_attempts: 2
    max_deep_attempts: 2

    allow_frontier: true

  signals:
    explicit_request: true
    repeated_tool_failure: true
    repeated_test_failure: true
    timeout: false
    backend_error: false
```



```
chain:
  - fast
  - deep
  - frontier

cloud:

  enabled: true

  allow_automatic_escalation: true

  max_requests_per_hour: 100

  max_estimated_cost_usd_per_day: 10.0

logging:
  level: "INFO"
  log_requests: true
  log_responses: false
  log_prompts: false

metrics:
  enabled: true
```

9. Secrets

Never store API keys in YAML.

OpenRouter key must come from:

```
OPENROUTER_API_KEY
```

Environment variables or a secure secret mechanism may be used.

Never print API keys.

Never include API keys in logs.

10. Automatic Routing

When:

```
model = auto
```

the initial route is:

```
fast
```

Do NOT invoke another LLM just to decide whether the task is hard.

The first version must use deterministic signals.

Initial behavior:

```
auto
↓
fast
```

Only escalate when configured escalation conditions occur.

11. Escalation Chain

The default escalation chain is:

```
FAST
↓
DEEP
↓
FRONTIER
```

Example:

```
Task starts
↓
local-fast
↓
two failed attempts
↓
local-deep
↓
two failed attempts
```

↓
frontier

The exact thresholds must be configurable.

12. Important: Model Failure vs Backend Failure

Distinguish these.

Backend failure

Examples:

```
connection refused
server unavailable
HTTP 500
HTTP 503
network failure
timeout
invalid backend response
```

These do NOT mean:

"The model couldn't solve the task."

Do not automatically escalate simply because llama-server crashed.

Handle backend failures using the configured backend fallback policy.

Task/model failure

Examples:

```
repeated tool failures
repeated test failures
explicit Claude escalation
repeated failed attempts
```

These may justify escalation.

13. Session State

Implement lightweight in-memory session state.

Client may provide:

```
X-LLM-Session-ID: <id>
```

If absent, generate one.

Track:

```
session_id  
current_route  
fast_attempts  
deep_attempts  
last_backend  
last_error  
escalation_count  
created_at  
updated_at
```

Do not store conversation content.

Do not persist prompts/responses by default.

14. Task State

Sessions and tasks are different concepts.

A session may contain multiple tasks.

Support an optional:

```
X-LLM-Task-ID: <id>
```

If absent, derive a reasonable request/task identifier.

Track per-task:

```
task_id
session_id
current_route
attempt count
failure signals
escalation history
timestamps
```

This allows:

```
Task A → escalates
Task B → remains fast
```

without contaminating the routing state of the whole session.

15. Explicit Escalation

Support an explicit client mechanism.

Header:

```
X-LLM-Escalate: deep
```

or:

```
X-LLM-Escalate: frontier
```

This must immediately select the requested tier, subject to cloud safety limits.

Also support:

```
X-LLM-Route: deep
```

for direct routing.

16. Claude Code Behavioral Policy

The router itself must not depend on CLAUDE.md.

However, provide documentation and an example CLAUDE.md policy that Claude Code users can install.

Example policy:

MODEL ESCALATION POLICY

Use the current local model for routine work.

Do not repeatedly attempt speculative fixes.

If you encounter:

- repeated test failures
- repeated tool failures
- difficult architectural decisions
- substantial uncertainty
- complicated cross-component interactions
- difficult debugging
- security-sensitive reasoning
- inability to explain why a proposed fix should work

request escalation to the deep model.

If the deep model cannot resolve the problem, request escalation to the frontier model.

When escalating, provide a short explanation of:

1. what has been attempted
2. what failed
3. what remains uncertain
4. what kind of reasoning is needed

This is a behavioral guideline for Claude Code.

It is NOT the authoritative escalation mechanism.

17. Claude Code Hooks

The system should support integration with Claude Code hooks through scripts/HTTP calls.

The router/controller may receive lifecycle signals such as:

```
tool execution
tool failure
test execution
test failure
task completion
task failure
```

Do not assume that every desired lifecycle event is directly available.

Implement a generic webhook endpoint:

```
POST /events
```

Example:

```
{
  "session_id": "abc",
  "task_id": "task-123",
  "event": "test_failure",
  "metadata": {
    "command": "pytest",
    "exit_code": 1
  }
}
```

Supported initial events:

```
task_start
task_complete
task_failure
tool_failure
test_failure
explicit_escalation
```

Unknown events must be accepted and logged at debug level rather than crashing.

18. Escalation Controller

Implement a separate component:

```
EscalationController
```

Its responsibility is to maintain task state and determine whether an escalation condition has been met.

Example:

```
class EscalationController:

    def record_event(...):
        ...

    def should_escalate(...):
        ...

    def current_route(...):
        ...

    def escalate(...):
        ...
```

The controller must NOT perform inference.

It only decides:

```
stay
escalate to deep
escalate to frontier
```

19. Objective Escalation Rules

Initial rules:

Rule A — explicit request

```
explicit deep request
  ↓
deep
```

```
explicit frontier request
  ↓
frontier
```

Rule B — repeated test failure

Example:

```
test failure
test failure
```

within the same task:

```
fast → deep
```

If the task is already on deep:

```
deep → frontier
```

Threshold must be configurable.

Rule C — repeated tool failure

Same concept.

Do not escalate on one isolated tool failure.

Default:

```
2 repeated relevant failures
```

Rule D — task retry count

If the same task repeatedly retries without success, escalation may occur.

Default:

```
fast: 2 attempts
deep: 2 attempts
```

20. No Self-Assessment Dependency

Do not require the local model to reliably determine:

"I am not smart enough."

Model self-assessment may be used as an optional future signal, but it must not be the foundation of v1.

The architecture must work without it.

21. Automatic Frontier Escalation

Cloud escalation must be opt-in.

Default:

```
allow_automatic_escalation: false
```

is recommended for the first deployment.

Once the system is trusted, it can be enabled.

If automatic cloud escalation is disabled:

```
deep failure
  ↓
do NOT silently call OpenRouter
  ↓
return escalation-required response
```

22. Cloud Cost Protection

Implement hard limits.

Example:

```
cloud:
  max_requests_per_hour: 100
  max_estimated_cost_usd_per_day: 10.0
```

If the limit is reached:

```
block automatic cloud calls
```

Manual explicit frontier requests may optionally have a separate policy.

Never create an uncontrolled cloud billing loop.

23. OpenRouter

Implement OpenRouter as an OpenAI-compatible backend.

Do not hard-code a model.

Configuration controls:

```
base URL
API key
model
timeouts
headers
```

Forward relevant OpenRouter/OpenAI-compatible request parameters without modification whenever possible.

24. Streaming

Streaming is mandatory.

For:

```
{  
  "stream": true  
}
```

the router must stream the response through to the client.

Do not buffer the entire response.

Preserve:

```
content chunks  
tool-call chunks  
reasoning chunks  
finish_reason  
usage where provided
```

25. Tool Calls

Tool calls are critical because Claude Code is an agent.

The router must transparently proxy:

```
tools  
tool_choice  
tool_calls  
function calls  
function arguments
```

Do not interpret or rewrite tool calls.

Do not alter tool schemas.

26. Context Preservation

The router must not summarize, truncate, rewrite, or otherwise alter the conversation in v1.

The complete request supplied by the client must be forwarded to the selected backend, subject only to backend/API requirements.

Context management is the client's responsibility.

27. Escalation Context

When escalating, do not automatically invent a new conversation.

The controller should expose escalation metadata so the client/Claude Code integration can decide how to invoke the stronger model.

Example:

```
{
  "escalation": {
    "from": "fast",
    "to": "deep",
    "reason": "repeated_test_failure",
    "attempts": 2
  }
}
```

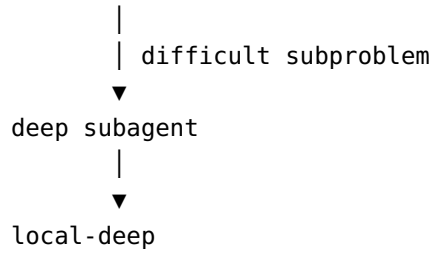
The preferred architecture is to use a **new deep-model request/subagent** where practical, rather than attempting to mutate the model underneath an already-running inference request.

28. Subagent-Friendly Design

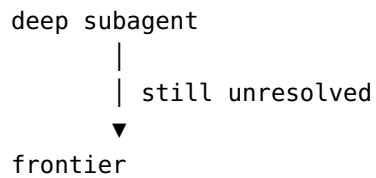
The router must support workflows where Claude Code delegates a difficult subtask to a stronger model.

Example:

```
Main Claude Code
  |
  ▼
local-fast
```



and:



The router must therefore work equally well for:

```
main agent
subagent
independent API client
```

29. Backend Interface

Create a common abstraction.

Example:

```
class Backend(ABC):

    async def chat_completion(
        self,
        request,
        *,
        stream: bool,
    ):
        ...

    async def health_check(self):
        ...
```

Implement:

```
OpenAICompatibleBackend  
OpenRouterBackend  
MockBackend
```

Do not put llama.cpp-specific logic in the routing layer.

`llama-server` is simply an OpenAI-compatible HTTP backend.

30. Health Checking

Implement:

```
GET /health
```

and:

```
GET /health/backends
```

Example:

```
{  
  "status": "ok",  
  "backends": {  
    "fast": {  
      "healthy": true,  
      "latency_ms": 32  
    },  
    "deep": {  
      "healthy": true,  
      "latency_ms": 68  
    },  
    "frontier": {  
      "healthy": true  
    }  
  }  
}
```

Do not make health checks expensive.

31. Backend Fallback

Backend failure and task escalation are separate.

Example:

```
fast backend unavailable
    ↓
if configured:
    deep
else:
    error
```

Do not automatically treat:

```
llama-server crashed
```

as:

```
model too weak
```

Cloud fallback must always be explicitly enabled.

32. Observability

Every request should have:

```
request_id
session_id
task_id
route
backend
timestamp
latency
status
escalation state
```

Example log:


```
request=abc123
session=session1
task=task7
route=fast
backend=local-fast
latency=4.82s
status=200
```

Escalation:

```
ESCALATION
session=session1
task=task7
from=fast
to=deep
reason=repeated_test_failure
count=2
```

Never log API keys.

Prompt and response logging must be disabled by default.

33. Metrics

Expose Prometheus-compatible metrics:

```
router_requests_total
router_request_latency_seconds
router_backend_requests_total
router_backend_errors_total
router_escalations_total
router_cloud_requests_total
router_cloud_blocked_total
router_tool_failures_total
router_test_failures_total
```

Labels should be kept low-cardinality.

Do not use raw session IDs as Prometheus labels.

34. CLI

Provide:

```
llm-router
```

and:

```
python -m router
```

Options:

```
--config config.yaml  
--host 127.0.0.1  
--port 8000  
--log-level INFO
```

35. Project Structure

Use approximately:

```
llm-router/  
|  
├─ pyproject.toml  
├─ README.md  
├─ config.example.yaml  
├─ .env.example  
├─ LICENSE  
|  
├─ router/  
|   ├─ __init__.py  
|   ├─ __main__.py  
|   ├─ server.py  
|   ├─ config.py  
|   ├─ api.py  
|   ├─ routing.py  
|   ├─ escalation.py  
|   ├─ sessions.py  
|   ├─ models.py  
|   └─ events.py
```

```
| | | metrics.py
| | | └─ backends/
| | |   | └─ __init__.py
| | |   | └─ base.py
| | |   | └─ openai_compatible.py
| | |   | └─ openrouter.py
| | |   └─ mock.py
| | |
| | └─ integrations/
| |   └─ claude_code/
| |     | └─ README.md
| |     | └─ escalation.sh
| |     └─ hooks/
| |
| └─ tests/
|   | └─ test_api.py
|   | └─ test_routing.py
|   | └─ test_escalation.py
|   | └─ test_sessions.py
|   | └─ test_streaming.py
|   | └─ test_tools.py
|   | └─ test_backends.py
|   └─ test_cloud_limits.py
```

36. Technology

Preferred:

```
Python 3.11+
FastAPI
httpx
Pydantic
PyYAML
asyncio
uvicorn
pytest
```

Keep dependencies minimal.

37. Mock Backend

A mock backend is mandatory.

The complete test suite must work without:

```
NVIDIA GPU  
llama-server  
OpenRouter  
Claude Code
```

The mock backend must simulate:

```
successful completion  
streaming  
tool calls  
backend errors  
timeouts
```

38. Testing

Test explicit routing:

```
local-fast → fast  
local-deep → deep  
frontier → OpenRouter  
auto → fast
```

Test escalation:

```
fast + failure → retry  
fast + threshold → deep  
deep + threshold → frontier
```

Test that backend errors do not incorrectly count as model-quality failures.

Test cloud limits.

Test session isolation.

Test task isolation.

Test streaming.

Test tool calls.

Test malformed requests.

Test unavailable backends.

Test that secrets never appear in logs.

39. Security

Bind to:

127.0.0.1

by default.

Do not expose the router publicly without explicit configuration.

Never log:

API keys
authorization headers

Prompt/response logging is disabled by default.

If authentication is later added, it must be optional for localhost deployments.

40. Systemd

Provide an example systemd service.

The service should:

- start automatically
- restart on failure
- load configuration from a specified path

- receive `OPENROUTER_API_KEY` securely
- write logs to journald

Do not make systemd mandatory for development.

41. Do NOT Implement in v1

Do NOT implement:

```
AI-based request classification
prompt rewriting
response rewriting
RAG
vector databases
web search
GUI
database
conversation summarization
GPU management
model downloading
model unloading
model loading
automatic llama-server restarts
```

These are future features.

42. Future: Intelligent Router

After deterministic routing is stable, optionally add an LLM classifier.

It may classify:

```
easy
medium
hard
frontier
```

But this must remain optional.

The deterministic policy must continue to work without the classifier.

Do not spend significant development effort on this in v1.

43. Future: GPU Lifecycle Manager

A future component may control llama-server:

```
start
stop
restart
load model
unload model
check VRAM
wait for readiness
```

This is explicitly out of scope for v1.

The initial system assumes:

```
llama-server :8001 = always running
llama-server :8002 = always running
```

44. Future: Adaptive Model Selection

Eventually the controller may maintain statistics such as:

```
task type
model
success rate
latency
tokens
cost
escalation rate
```

This could enable policies such as:

```
"Python test fixes are almost always solved by fast."
```

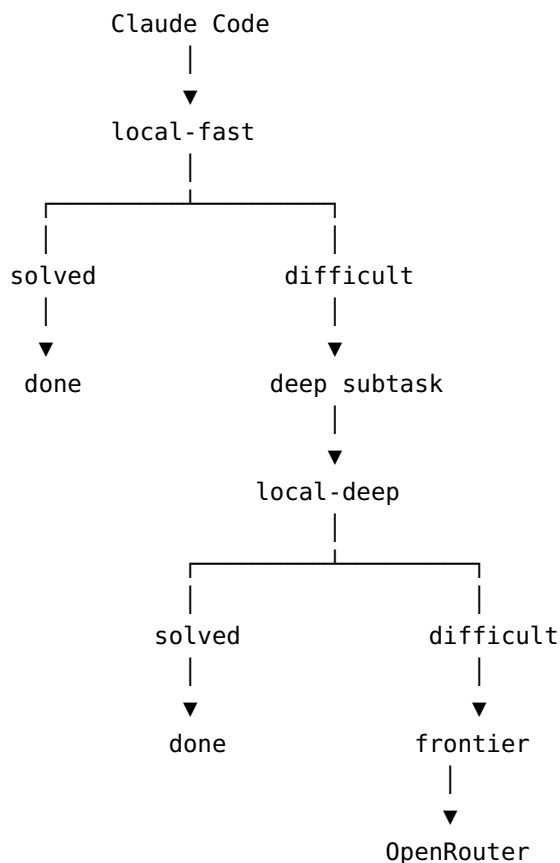
or:

"Large architectural tasks should go directly to deep."

Do not implement this learning system in v1.

45. Recommended Claude Code Workflow

The intended workflow is:



The main Claude Code agent should remain responsible for the overall task.

Stronger models should preferably be used for difficult subtasks/reasoning rather than blindly restarting the entire conversation at a different model.

46. Manual Controls

Provide simple manual controls.

At minimum:

```
X-LLM-Route: fast
X-LLM-Route: deep
X-LLM-Route: frontier
X-LLM-Route: auto
```

and:

```
X-LLM-Escalate: deep
X-LLM-Escalate: frontier
```

Document how these can be used from shell scripts and Claude Code hooks.

47. Example Event Flow

Example difficult coding task:

1. Claude Code starts task.
2. Request goes to:
local-fast
3. Claude edits code.
4. Tests fail.
5. Claude attempts correction.
6. Tests fail again.
7. Claude Code hook emits:
test_failure
8. Controller sees:
task=123
fast_failures=2
9. Controller changes route:
fast → deep
10. Claude/deep subagent receives the difficult reasoning task.

11. Deep model solves it.
12. Main Claude Code continues.
13. No cloud request is made.

If deep also fails:

```
deep_failures=2
  ↓
frontier
  ↓
OpenRouter
```

provided cloud escalation is enabled.

48. Key Architectural Constraint

Do not implement:

```
Claude Code
  ↓
router
  ↓
kill llama-server
  ↓
load another model
  ↓
continue
```

in v1.

Instead:

```
Claude Code
  ↓
router/controller
  |— fast llama-server
```

```
|— deep llama-server
|— OpenRouter
```

Keep the models hot.

The user's hardware has sufficient VRAM to make this architecture practical.

49. Deliverables

Produce:

1. Complete Python source.
2. `pyproject.toml`.
3. Example YAML configuration.
4. `.env.example`.
5. Unit tests.
6. Mock backend.
7. README.
8. Claude Code integration documentation.
9. Example Claude Code CLAUDE.md escalation policy.
10. Example Claude Code hook scripts.
11. Example systemd service.
12. Example llama-server commands showing how the two local backends can be started.
13. Troubleshooting documentation.

50. Definition of Done

The project is considered complete when the following works:

```
Claude Code
  ↓
local router :8000
  ↓
local-fast  :8001
```

and:

```
Claude Code
  ↓
local router :8000
```

```
↓  
local-deep :8002
```

and:

```
Claude Code  
↓  
local router :8000  
↓  
OpenRouter
```

and:

```
Claude Code  
↓  
auto  
↓  
fast  
↓  
repeated task failure  
↓  
deep  
↓  
repeated task failure  
↓  
frontier
```

with:

- streaming
- tool calls
- session/task tracking
- deterministic escalation
- health checks
- metrics
- cloud budget protection
- no model swapping
- no prompt logging by default
- no API-key leakage
- complete automated tests

The implementation should be understandable by one developer and easy to modify.

Do not add functionality that is not required by this specification.